SERVICIO NACIONAL DE APRENDIZAJE - SENA

Centro de Formación en Gestión y Servicios

Servicios web, creación API

Autor: Nestor Alfredo Rubiano Torres

Programa: Análisis y desarrollo de software

Código de Ficha: 3118311

Instructor: Andrés Rubiano Cucarían

Actividad: GA7-220501096-AA5

Ciudad: Bogotá, Colombia

Introducción

El presente documento detalla el diseño arquitectónico y técnico de los servicios web (API REST) desarrollados para el proyecto formativo "Madhouse Barbería". La construcción de esta API se ha realizado utilizando el framework Spring Boot (Java), garantizando una comunicación eficiente y segura mediante el formato de intercambio de datos JSON y la implementación de encriptación BCrypt para la protección de credenciales.

El diseño contempla los módulos mas importantes del sistema: Autenticación, Gestión de Usuarios (diferenciando roles entre Clientes y Barberos), Catálogo de Servicios y Operación de Reservas. Siendo un plano de construcción para la integración exacta con el frontend en React JS desarrollado en fases anteriores.

Objetivo

Diseñar y documentar la arquitectura de los servicios web tipo RESTful necesarios para el funcionamiento del software "Madhouse Barbería", utilizando estándares de la industria y garantizando la integridad referencial de la base de datos.

Estructurar el modelo de datos para la gestión unificada de usuarios, aplicando filtrado lógico por el atributo rol.

Definir los Endpoints para la autenticación segura (Login) y registro, aislando esta lógica en un controlador específico (AuthController).

Diseñar las rutas HTTP (GET, POST, PUT) para los flujos operativos, incluyendo la edición de perfiles, cambio seguro de contraseñas y consulta de catálogo.

Arquitectura de la API REST

La API está construida bajo el patrón arquitectónico adaptado para servicios REST en Spring Boot, estructurado en las siguientes capas:

Controllers: Exponen las rutas web (Endpoints) y reciben las peticiones HTTP (AuthController, UsuarioController, ServicioController, ReservaController).

DTOs (Data Transfer Objects): Objetos de transferencia implementados para recepcionar estructuras complejas (ej. ReservaRequest que unifica la entidad Reserva con la lista de IDs de servicios elegidos).

Repositories (Data Access): Interfaces que extienden de JpaRepository para el manejo automatizado de consultas SQL, incluyendo *Derived Queries* (ej. findByCorreo).

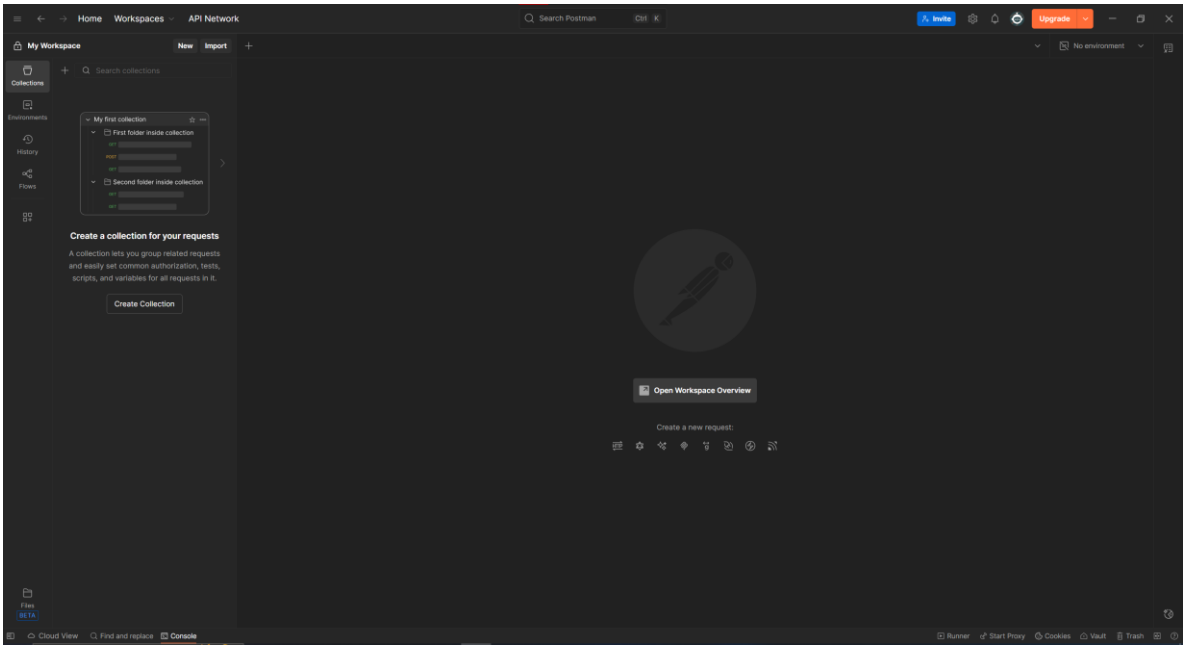
Models (Entidades): Representaciones exactas de las tablas de la base de datos MySQL (Usuario, Servicio, Reserva, ReservaServicio).

Diseño Estructural

- **Gestión Unificada de Usuarios:** Tanto Clientes como Barberos viven en la tabla usuario. La diferenciación operativa se realiza mediante la columna rol (ROLE_CLIENTE o ROLE_BARBERO).
- **Seguridad (Hashing):** Las contraseñas no viajan ni se almacenan en texto plano. Se emplea BCryptPasswordEncoder de Spring Security para el registro y la validación en el login.
- **Manejo de Tiempos:** Para optimizar la interacción con interfaces gráficas web el modelo Reserva utiliza java.time.LocalTime apoyado por la anotación @JsonFormat(pattern = "HH:mm").

Documentación Técnica de Endpoints (Diseño de API)

A continuación, se detallan los servicios web diseñados y codificados para cubrir las características del proyecto formativo con ayuda de la herramienta Postman



Módulo 1: Autenticación y Seguridad (AuthController)

Registro de Usuario (Registro)

Método HTTP: POST

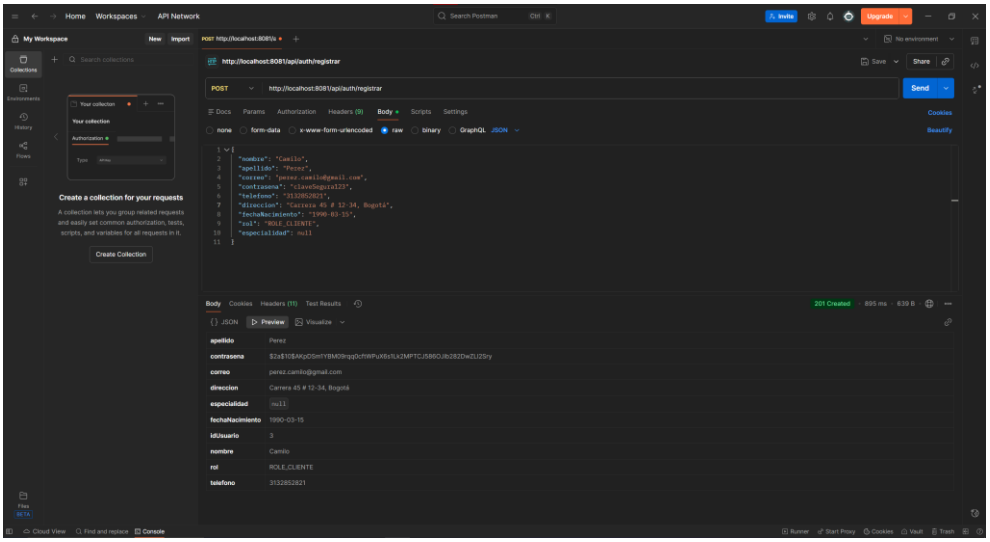
Ruta (Endpoint): /api/auth/registrar

Descripción: Crea un nuevo usuario en la base de datos aplicando hashing a la contraseña. Asigna ROLE_CLIENTE por defecto si no se especifica.

Body Request (JSON):

JSON

```
{
  "nombre": "Camilo",
  "apellido": "Perez",
  "correo": "perez.camilo@gmail.com",
  "contrasena": "claveSegura123",
  "telefono": "3132852821",
  "direccion": "Carrera 45 # 12-34",
  "fechaNacimiento": "1990-03-15",
  "rol": "ROLE_CLIENTE"
}
```



Response (201 Created / 200 OK): Retorna el objeto del usuario creado con la contraseña encriptada y su idUsuario autogenerado.

Iniciar Sesión (Login)

Método HTTP: POST

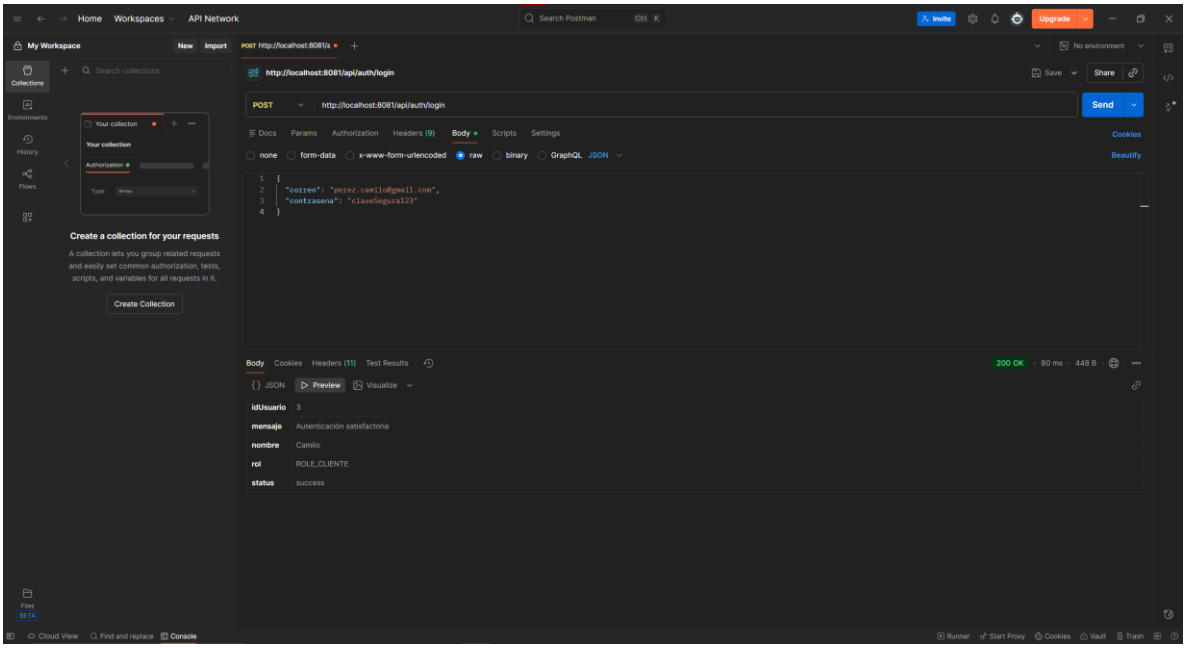
Ruta (Endpoint): /api/auth/login

Descripción: Valida la existencia del correo y compara el hash de la contraseña.

Body Request (JSON):

JSON

```
{
  "correo": "perez.camilo@gmail.com",
  "contrasena": "claveSegura123"
}
```



- **Response Éxito (200 OK):**

JSON

```
{
```

"status": "success",

"mensaje": "Autenticación satisfactoria",

"idUsuario": 1,

"nombre": "Camilo",

"rol": "ROLE_CLIENTE"

}

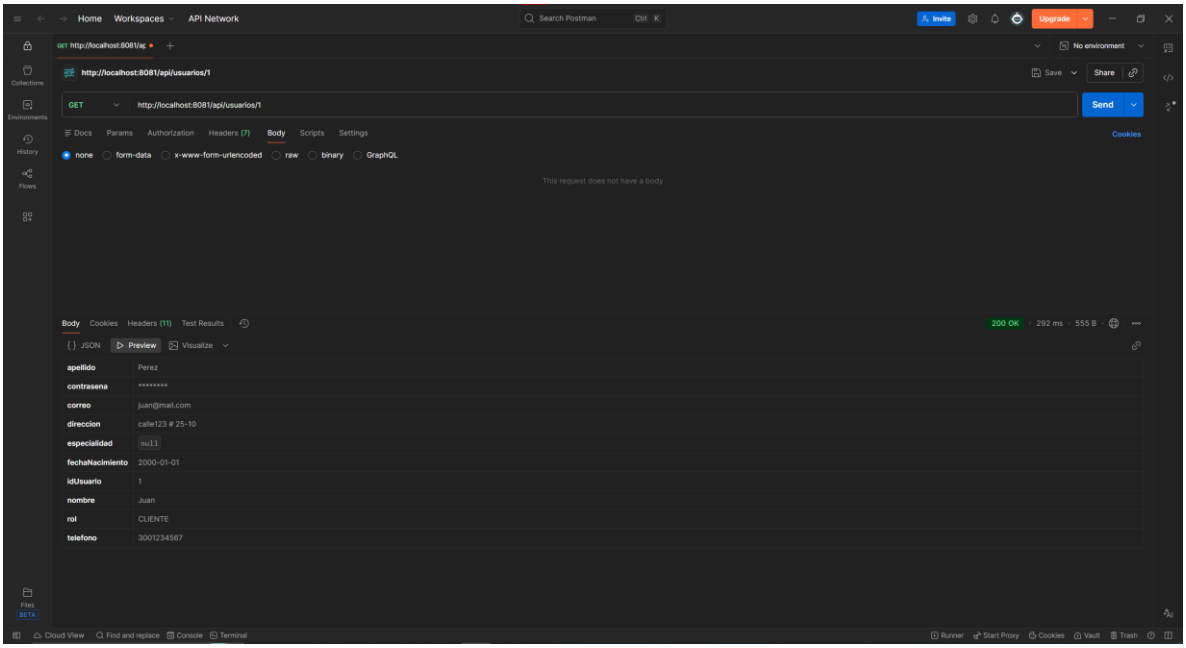
Módulo 2: Gestión de Perfiles (UsuarioController)

Obtener Datos del Perfil

Método HTTP: GET

Ruta (Endpoint): /api/usuarios/{id}

Descripción: Retorna la información completa de un usuario por su ID para llenar la vista "Mi Perfil".



Actualizar Información Básica

Método HTTP: PUT

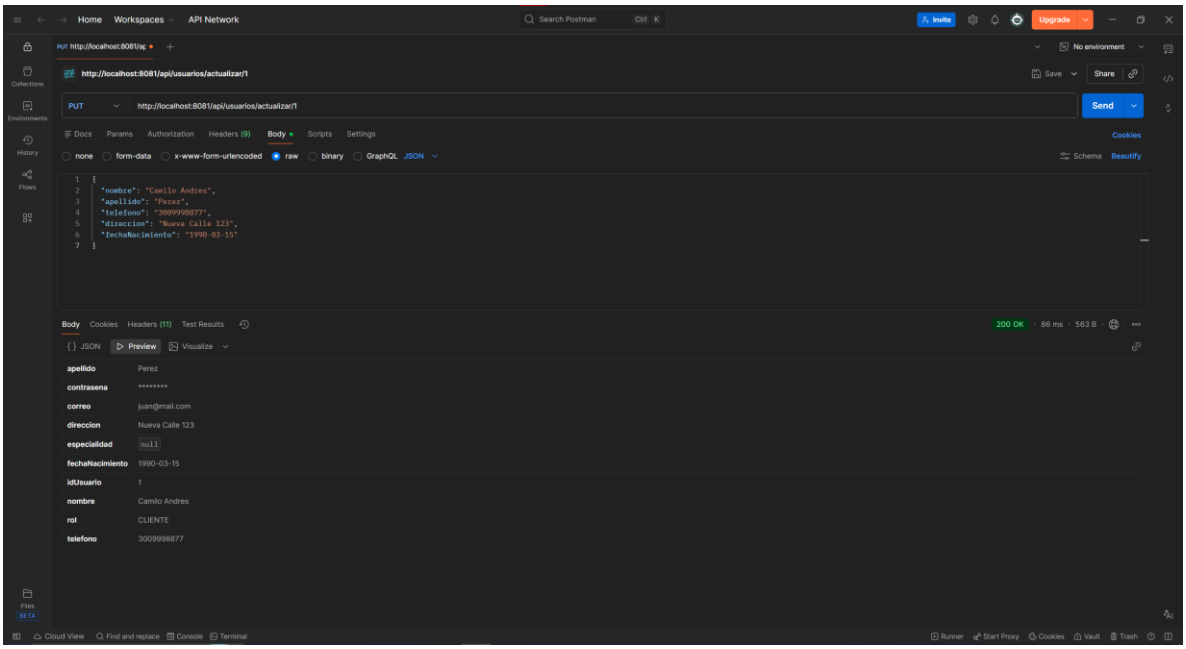
Ruta (Endpoint): /api/usuarios/actualizar/{id}

Descripción: Permite modificar datos personales excluyendo variables sensibles (correo, contraseña, rol).

Body Request (JSON):

JSON

```
{
  "nombre": "Camilo Andres",
  "apellido": "Perez",
  "telefono": "3009998877",
  "direccion": "Nueva Calle 123",
  "fechaNacimiento": "1990-03-15"
}
```

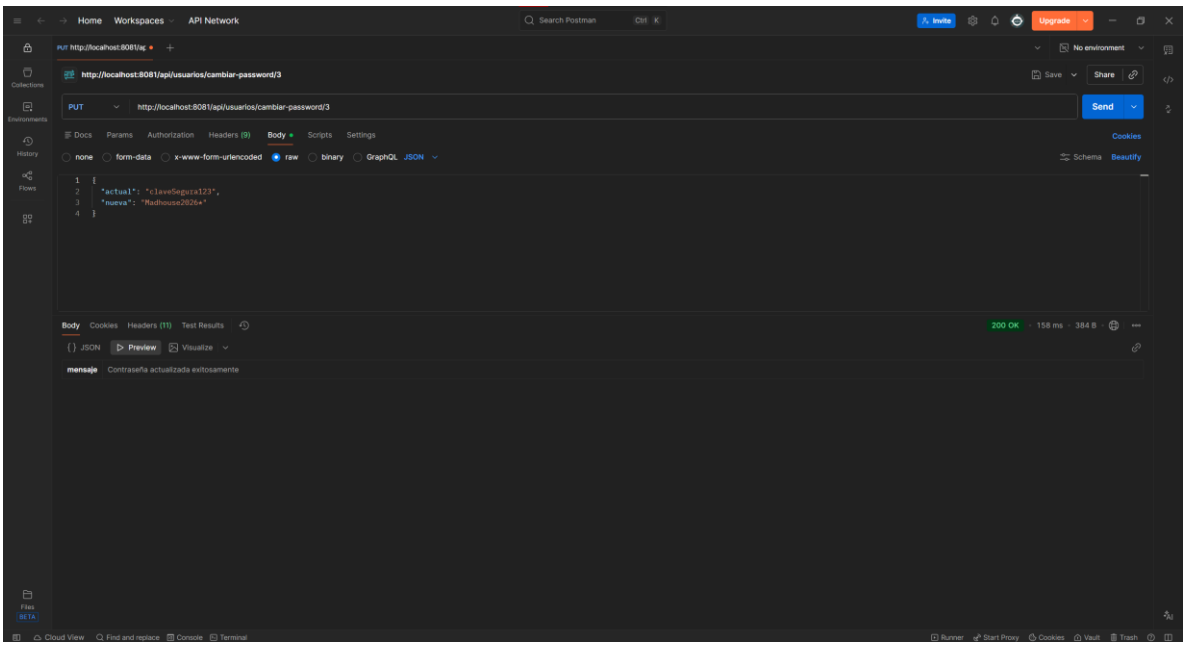


Cambiar Contraseña Segura

- **Método HTTP:** PUT
- **Ruta (Endpoint):** /api/usuarios/cambiar-password/{id}
- **Descripción:** Verifica la contraseña actual ingresada y, de coincidir, guarda el hash de la nueva contraseña.
- **Body Request (JSON):**

JSON

```
{
  "actual": "claveSegura123",
  "nueva": "Madhouse2026*"
}
```

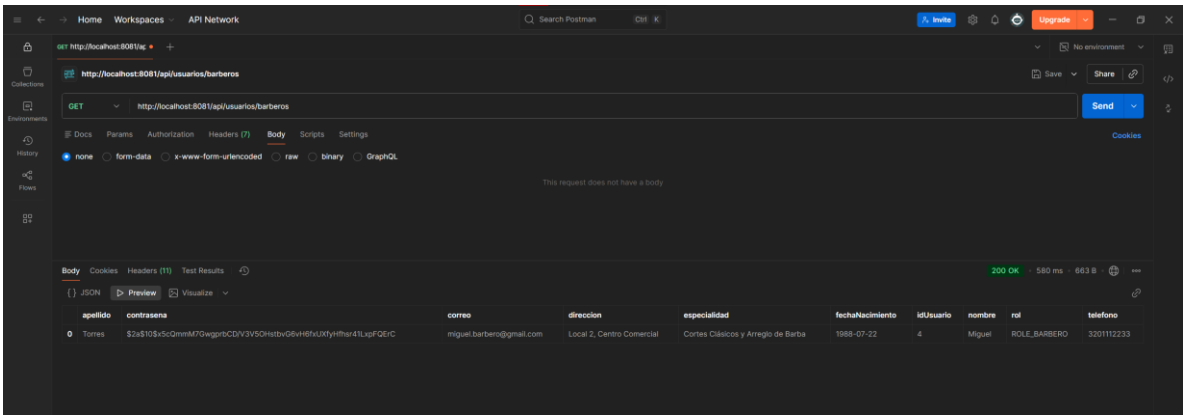


Catálogo de Barberos

Método HTTP: GET

Ruta (Endpoint): /api/usuarios/barberos

Descripción: Retorna una lista exclusiva de usuarios filtrada por rol = 'ROLE_BARBERO'.



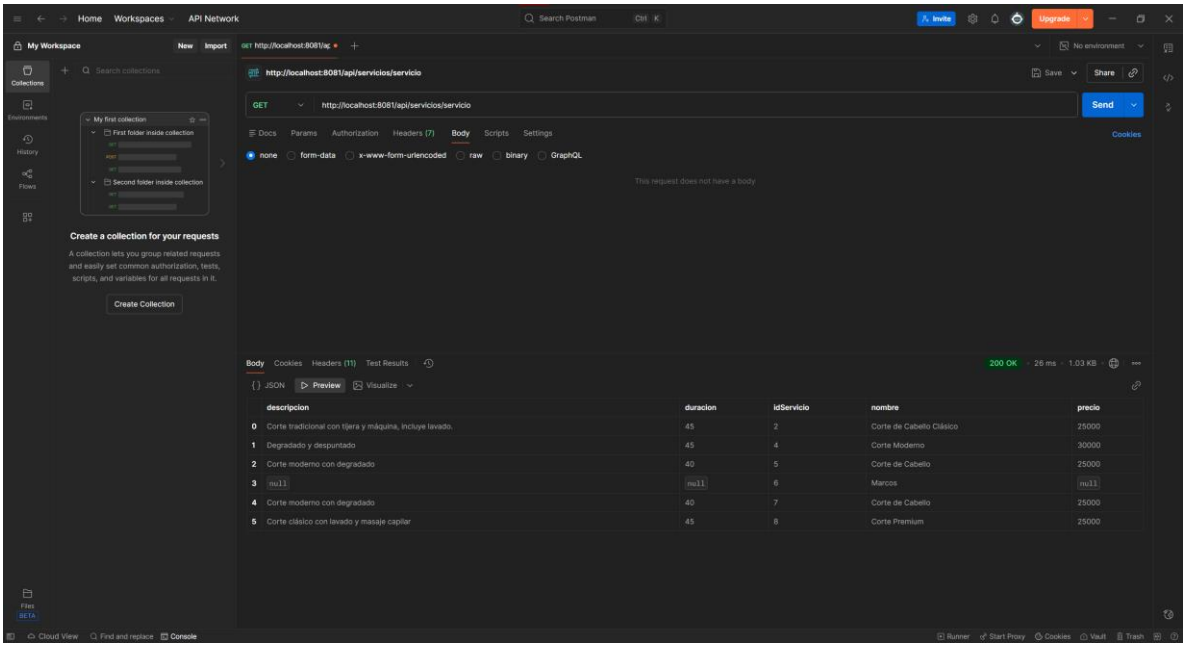
Módulo 3: Catálogo y Operación (ServicioController y ReservaController)

Listar Servicios de Barbería

Método HTTP: GET

Ruta (Endpoint): /api/servicios/servicio

Descripción: Obtiene el catálogo completo de servicios (nombre, descripción, precio, duración) para el *wizard* de reservas.



Crear Nueva Reserva (Flujo Completo)

Método HTTP: POST

Ruta (Endpoint): /api/reservas/crear

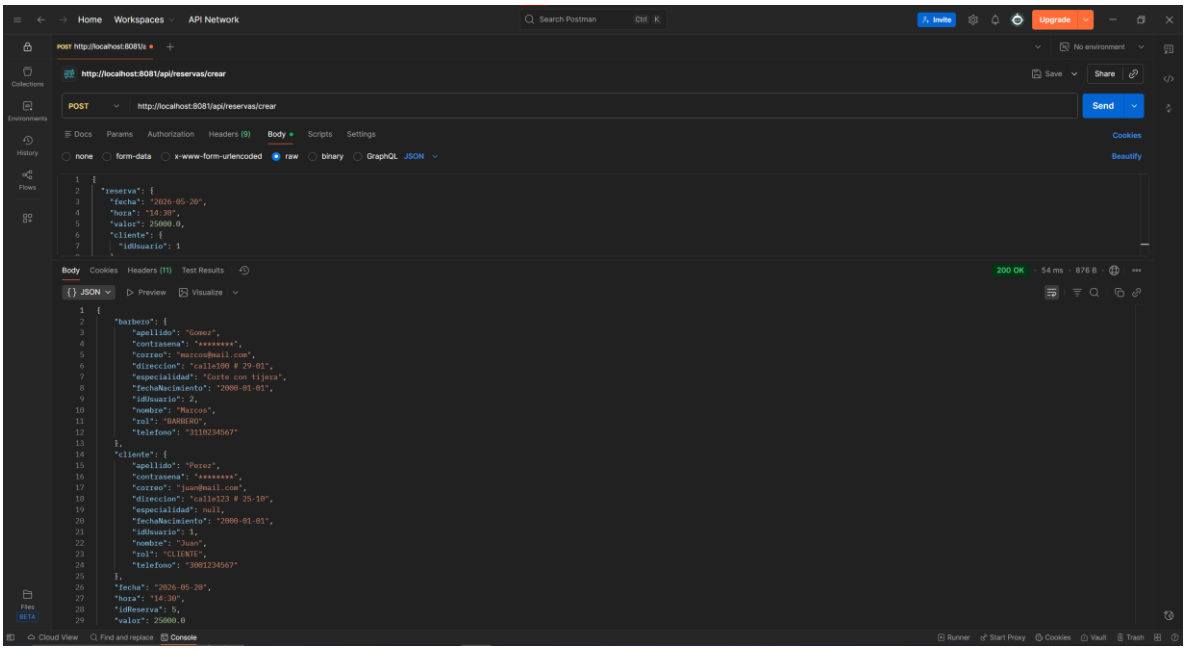
Descripción: Recibe un DTO (ReservaRequest) que contiene la información cabecera de la cita y un arreglo con los IDs de los servicios elegidos. El backend se encarga de instanciar la relación en la tabla detalle ReservaServicio.

Body Request (JSON):

JSON

```
{
  "reserva": {
    "fecha": "2026-05-20",
    "hora": "14:30",
    "valor": 25000.0,
    "cliente": {
      "idUsuario": 1
    },
    "barbero": {
      "idUsuario": 2
    }
  },
  "idServicios": [1]
}
```

Response (200 OK): Retorna el objeto Reserva consolidado con sus relaciones JPA mapeadas.

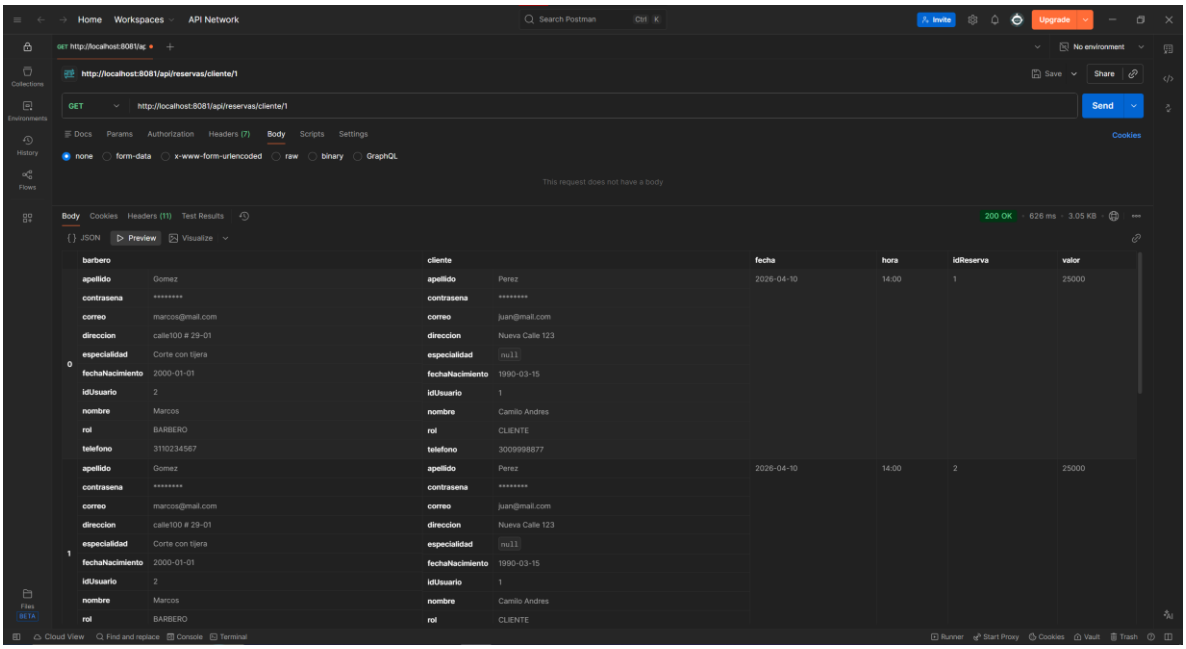


Consultar Reservas de un Cliente

Método HTTP: GET

Ruta (Endpoint): /api/reservas/cliente/{clienteId}, /api/reservas/barbero/2

Descripción: Retorna el historial de citas asociadas a un cliente o a un barbero específico para llenar la vista de "Mis Reservas".



Versionamiento

Enlace del repositorio remoto de GitHub: **https://github.com/NestorVal/Madhouse-app/tree/main/backend**

Video

Verificacion de endpoints: https://www.youtube.com/watch?v=A998j_4nHtQ

Archivos

Dentro de la carpeta backend se encuentra el pdf correspondiente a la creacion de servicios web, archivo JSON con la colección de Endpoints.postman y los archivos correspondientes a la API de Madhouse.

Conclusión

El diseño y desarrollo de la API REST para "Madhouse Barbería" establece una arquitectura robusta, segura y alineada con los requerimientos del frontend. La implementación de Data Transfer Objects (DTOs) optimiza la recepción de transacciones complejas, como la creación de reservas con múltiples servicios asociados. Asimismo, la configuración de seguridad con BCrypt y el manejo modernizado de fechas y horas (LocalTime) aseguran que el sistema sea escalable, mantenible y cumpla con las directrices técnicas exigidas para la fase de construcción de servicios web del proyecto formativo.